

DATAVINCI: Learning Syntactic and Semantic String Repairs

MUKUL SINGH, Microsoft, USA

JOSÉ CAMBRONERO, Microsoft, USA

SUMIT GULWANI, Microsoft, USA

VU LE, Microsoft, USA

CARINA NEGREANU, Robin AI, UK

ARJUN RADHAKRISHNA, Microsoft, USA

GUST VERBRUGGEN, Microsoft, Belgium

String data is common in real-world datasets: 67.6% of values in a sample of 1.8 million real Excel spreadsheets from the web were represented as text. Automatically cleaning such string data can have a significant impact on users. Previous approaches are limited to error detection, require that the user provides annotations, examples, or constraints to fix the errors, and focus independently on syntactic errors or semantic errors in strings, but ignore that strings often contain both syntactic and semantic substrings. We introduce DATAVINCI, a fully unsupervised string data error detection and repair system. DATAVINCI learns regular-expression-based patterns that cover a majority of values in a column and reports values that do not satisfy such majority patterns as data errors. DATAVINCI can automatically derive edits to the data error based on the majority patterns and using row tuples associated with majority values as examples. To handle strings with both syntactic and semantic substrings, DATAVINCI uses an LLM to abstract (and re-concretize) portions of strings that are semantic. Because not all data columns can result in majority patterns, when available, DATAVINCI can leverage execution information from an existing data program (which uses the target data as input) to identify and correct data repairs that would not otherwise be identified. DATAVINCI outperforms eleven baseline systems on both data error detection and repair as demonstrated on four existing and new benchmarks.

CCS Concepts: • **Information systems** → **Data cleaning**; *Language models*; • **Computing methodologies** → **Information extraction**.

Additional Key Words and Phrases: Data Cleaning, Data Error Detection, Semantic Data Errors.

ACM Reference Format:

Mukul Singh, José Cambroneró, Sumit Gulwani, Vu Le, Carina Negreanu, Arjun Radhakrishna, and Gust Verbruggen. 2025. DATAVINCI: Learning Syntactic and Semantic String Repairs. *Proc. ACM Manag. Data* 3, 1 (SIGMOD), Article 27 (February 2025), 26 pages. <https://doi.org/10.1145/3709677>

1 Introduction

Errors in tabular data, such as inconsistent or corrupted values, are common and can result in incorrect computations, invalid conclusions, and downstream data pipeline failures [8]. Such data errors can stem from a variety of sources including manual data entry, data integration or merging,

Authors' Contact Information: Mukul Singh, singhmukul@microsoft.com, Microsoft, Redmond, USA; José Cambroneró, jcambroneró@microsoft.com, Microsoft, Redmond, USA; Sumit Gulwani, sumitg@microsoft.com, Microsoft, Redmond, USA; Vu Le, levu@microsoft.com, Microsoft, Redmond, USA; Carina Negreanu, csnegreanu@gmail.com, Robin AI, Cambridge, UK; Arjun Radhakrishna, arradha@microsoft.com, Microsoft, Redmond, USA; Gust Verbruggen, gverbruggen@microsoft.com, Microsoft, Keerbergen, Belgium.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2836-6573/2025/2-ART27
<https://doi.org/10.1145/3709677>

Excel				
Phone	Code	Gender	Code	Website
937-587-3389	US-837	Male	M-432A	https://www.google.com
419-996-7110	UK-828	Male	M-233-A	https://www.youtube.com
440-993-8351	US-219	Female	F-322R	www.facebook.com
(937) 509 6413	us291	Male	M-310R	https://www.twitter.com
937-587-3389	POL-921	Female	F-275D	https://www.instagram.com
419-908-7099	AUS-463	Female	F-283R	https://www.linkedin.com
4405764039	USA-392	Male	435d	https://www.reddit.com
419-6310-239	UK-389	Male	M-384A	https://www.amazon.com

Wikipedia			
Mixing	Length	Area	Units
47 (7.55%)	04:34	Birmingham	12.0 oz
38 (6.09%)	05:23	London	12.0 OZ
26 (4.24%)	04:38	Cardiff	12.0 oz
21 (3.42%)	03.45	Hampton	12.0 oz
19 (3.05%)	03:34	Rockford	12.0 ounces
16 (2.66%)	04:55	Birmingxham	12.0 oz
19 (3.05%)	03:22	London	12.0 oz
16 (2.66%)	04:12	Wales	12.0 oz

Fig. 1. Examples of string data errors found in Wikipedia tables and Excel spreadsheets. The data errors in each column are highlighted. It shows that there is a mix of both syntactic (like "03.45", "21 (3.42%)") and semantic data errors (like "Birmingxham" and "12.0 ounces") found in real world tables.

and faulty computations or transformations. Prior work [36] reported that even in professional settings, such as financial institutions and consulting firms, up to 24% of spreadsheets have at least one mistake, including data errors. Figure 1 shows some examples of errors found in online Wikipedia tables and public Excel spreadsheets.

In a sample of 1.8 million Excel spreadsheets from the web, we found that 67.6% of values are represented as text, providing a substantial opportunity for string repair systems. While prior work has introduced approaches to repair string data, these typically have focused primarily on detecting errors but not repairing them [10, 11, 26, 36], required users to provide (partial) annotations or constraints [17, 18, 30, 31], or have relied on a limited rule-learning approach [1]. Furthermore, effective string cleaning must support errors values consisting of both syntactic and semantic substrings. For example, given a column with three values [("NY", "(Boston)", "(Miami)"] where the pattern is both semantic (city names) and syntactic (parenthesized values), a repair system should report the first entry as a data error and suggest "(New York)" as the repaired value. Unfortunately, prior rule-based and external-knowledge-based systems can only tackle the syntactic issue or the semantic issue, respectively, but not the combination of these.

We introduce DATAVINCI, a fully automated error detection and resolution system for string columns in tabular data. In contrast to prior work [11, 18, 36], DATAVINCI does not only detect data errors in strings but also suggests repairs. To our knowledge, DATAVINCI is the first system that detects and repairs errors in strings that consist of both syntactic and semantic substrings—prior work has been tailored to either category separately. Finally, DATAVINCI can perform detection and repair in a fully unsupervised manner, without requiring user inputs such as providing constraints [10], examples [18], or annotations [30].

To perform fully unsupervised error detection on a string column, DATAVINCI exploits the regularity in a column of string data, and reports as data errors those values that do not satisfy patterns associated with a (configurable) large fraction of the column's values. In contrast to existing pattern-based work [1, 11, 26], DATAVINCI uses a large language model to identify and mask semantic substrings, allowing the regular-expression-based pattern learner to capture patterns that can identify strings with both syntactic and semantic substrings.

While prior string repair systems require that the user provide examples [17], annotations [30], or constraints [31], DATAVINCI can suggest data repairs without any additional input. Specifically, DATAVINCI generates candidate repairs by deriving a minimal sequence of edit actions that can be applied to an erroneous value to satisfy a significant pattern. These edits can introduce holes for semantic values that must be concretized to produce a repaired value. DATAVINCI takes the observed substrings associated with a hole as outputs and the row tuples as inputs and learns a prediction function to fill each such hole separately. DATAVINCI sorts the final set of candidate repairs based on a heuristic ranker to compute likelihood of the repair being correct.

We evaluate DATAVINCI on an existing benchmark (Wikipedia web tables [11]) and three new benchmarks (a collection of real-world tables found in public Excel spreadsheets, synthetically corrupted spreadsheet tables, and Excel formulas and their input spreadsheet data). We compare our performance to 7 baselines, which include 5 existing error detection/repair systems, one LLM-based baseline, and one small transformer-based trained model. In addition, we augment detection-only systems with a GPT-3.5 error repair module to extend these to repair. Jointly these cover a variety of existing approaches from the data management community or reasonable alternatives.

We find that DATAVINCI's fully unsupervised pattern-based detection and repair approach leads to higher precision detection (+1.6 to +8.7 points than next best), higher recall detection (+3.6 than next best), and higher precision repairs (+1.6 to +12.2 points than next best). We find that removing the LLM-based abstraction and concretization approach that enables repairing mixed syntactic and semantic strings reduces repair precision and recall (-3.8 and -6 points, respectively).

Additionally, we perform a case study on *execution-guided repair*. Real data may not always display significant patterns. For example, consider a column *col1* with values ["c-1", "c-2", "c3", "c4"]. A pattern learner would identify two patterns: one for the first two and one for the last two values. Without further information, it is not possible to identify any of these entries as a data error. However, executing the user-written spreadsheet formula =SEARCH("-", [@col1]), which searches for the "-" in a string, on the first two values will yield a valid result, but on the last two values will result in an exception. DATAVINCI can exploit this execution information to suggest repairs that allow successful execution of the formula. We show that applying DATAVINCI's execution-guided repair to our benchmark of Excel formulas raises formula-level execution success rates to 54% (single-column inputs) and 47.8% (multi-column inputs) compared to 43.2% and 35.7% for DATAVINCI without execution-guided repair, and substantially outperforms the best non-DATAVINCI baseline.

To summarize, our contributions are:

- We develop a pattern-based approach, called DATAVINCI, to detect *and* repair errors in strings with both syntactic and semantic substrings without any supervision.
- We carry out an extensive evaluation of DATAVINCI on multiple datasets from different domains and against multiple existing systems, which shows that DATAVINCI can outperform existing string error detection and repair approaches.
- We introduce execution-guided repair for data cleaning, where DATAVINCI executes an existing program that depends on the target column and uses the resulting execution success information for individual values to learn patterns, thereby reducing the associated program failures by applying repairs dedicated to the column errors.

- We release the URLs and data preparation scripts to produce our three novel Excel-based benchmarks.¹ Further, we release annotations for DATAVINCI on these benchmarks.

2 Problem Statement

We take inspiration from the data error detection and repair formulation presented in prior work [11, 31]. First, we introduce our target domain of string data errors.

Let T be a table, consisting of a collection of m columns $\{c_1, \dots, c_m\}$, each with n values. Let c_i be a string column consisting of the *observed values* $\{v_1, \dots, v_n\}$, where each v_j is a string. For each v_j , let v_j^* be the *latent clean value*. If $v_j \neq v_j^*$, we say v_j is a string data error. The goal of a string repair system is not only to identify such string errors, but also provide repaired values.

Let v be a string value with a data error ($v \neq v^*$) in a column c of table T . Let $\hat{v}$ be a string value produced by a data repair system, given v and the table T . We say $\hat{v}$ is a candidate string error repair. If $\hat{v} = v^*$, we say it is a successful string error repair.

EXAMPLE 1. In Figure 1, the highlighted strings correspond to string data errors. For example, in the *Mixing* column, the value "21 (3.42%" is the observed value, while the corrected version "21 (3.42%)" is the latent clean value. Intuitively, the error here is syntactic—that is, the data error can be corrected by only examining the format of the string. On the other hand, the data error in the string "Birminxham" of the *Area* column is semantic. On a purely syntactic level, the incorrect value has the same format as all the correct values in that column. However, to produce the correction *Birmingham*, a string error correction system needs to understand that the values from the *Area* column refer to cities.

REMARK 1. The category of errors we consider in this paper are within-column—not arising from the relationship between different columns of the same record. Hence, this excludes a variety of errors that are covered by and detectable using techniques that discover (conditional) functional dependency, denial constraints, and other integrity constraints [1, 11]. Instead, we focus on pattern-based errors within a single column of string values—where most strings in the column fall into a certain pattern (in our case, a semantic regular expression), but the erroneous values do not. We use the terms *syntactic* and *semantic* specifically in this pattern-based error detection context as well. The term *semantic* does not refer to the semantic relationship between different values, but the inherent meaning of the sub-string.

3 DATAVINCI

An overview of DATAVINCI is shown in Figure 2 and summarized here. Each component is detailed in the sections below.

①–② Given a column c , we first learn the *primary patterns* $r_1, r_2, \dots, r_k$ in the column c , as well as the outliers. We use the term *primary pattern* to refer to each of r_i —the full semantic regular language is given by the alternation $r_1 | \dots | r_k$. The primary patterns for column *Player ID* are Country-[0-9]+-CAT1 and QUAL-[0-9]+ where CAT1 represents the categorical datatype JUN|PRO, and the sole outlier is the string "austria_837". For each outlier, we then repair the string assuming that the corrected value will match the primary pattern r_i .

③ Each repair is done by first generating a *repair sketch*—a string with empty holes that need to be filled. In Figure 2, the edit sketches "QUAL-837" and "US-837-{CAT1}" correspond to the primary patterns QUAL-[0-9]+ and Country-[0-9]+-{CAT1}. The second sketch has two holes (Country and {CAT1}) which needs to be filled.

④–⑤ Holes in repair sketches are filled to obtain concrete edit scripts. For syntactic categories, we learn value constraints on the equivalent sub-strings in the non-outlier rows of the table (like

¹Due to compliance we are not able to release the post-processed data, so we release scripts replicate our benchmarks.

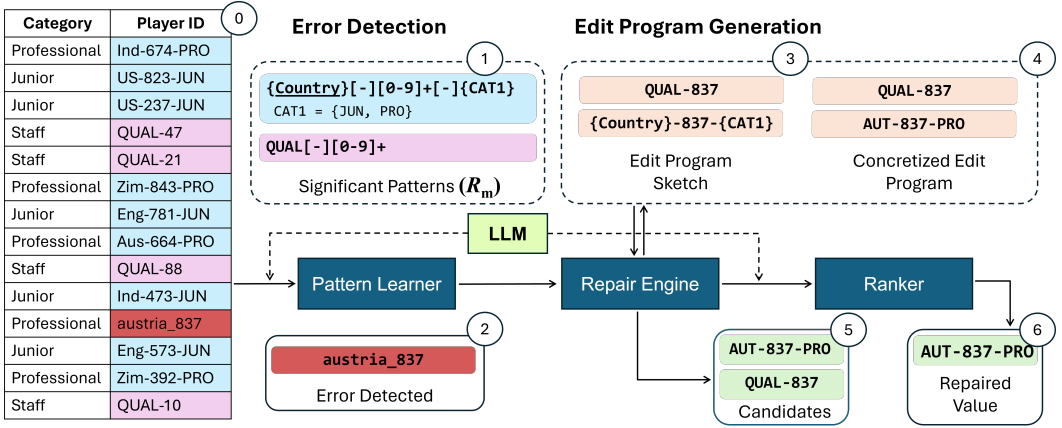


Fig. 2. DATAVINCI illustrated with an example from our corpus. ① denotes the input table. ① DATAVINCI learns significant patterns, which account for both syntactic and semantic substrings, for the *Player ID* column. The values satisfying each pattern are shown in the pattern's color. Underlined pattern elements (Country) correspond to LLM-abstracted semantic substrings. ② Significant patterns are used to detect outliers: "austria_837" is detected as an error as it does not satisfy a significant pattern. ③ Edit programs for the outlier are learned by deriving edits to the string that will satisfy a significant pattern. ④ Actions in edit programs that operate on abstractions (wrapped in { }) are concretized by looking at other rows and learning decision trees (syntactic, CAT1 → "PRO") or using the LLM (semantic, Country → "AUT"). ⑤ Applying these value constraints produces the candidate repairs. ⑥ The heuristic ranker sorts candidates and the top ranked candidate is suggested to the user.

(*Category* = "Professional") → "PRO"). For semantic categories, we leverage an LLM (Country → "AUT").

⑥ Candidates are ranked to obtain the final repair "AUT-837-PRO".

3.1 Generalizing Strings to Patterns

We represent patterns in the string values of a column using semantic regular expressions [6]. However, we restrict ourselves to patterns represented semantic regular expressions that allow for feasible data repairs. Formally, we define a *pattern* as

```
pat := str | charClass | charClass*
      | pat · pat | semToken[fmt]
```

The constructs `str`, `charClass`, `*`, and `·` represent constant strings, character classes, Kleene stars, and concatenation, respectively, with each having their usual semantics. Note that we only allow for Kleene stars on character classes, and not on more complex expressions. This restricted fragment of regular expressions has been used in several prior works to represent patterns in string data in columns [24].

The non-standard construct in semantic regular expressions is the *semantic token* construct `semToken[fmt]`. Semantic tokens represent entities from a known set of semantic categories such as names, addresses, countries, currency, and so on. They are shown underlined in expressions. The optional `fmt` represents the formatting of the entity. As an example, the token Country[full] will match the strings "United Kingdom" and "Hungary", while the token Country[abbrev] will match the strings "UK" and "HU". In the rest of the paper, we often omit the `[fmt]` from the token when the formatting is clear from the context.

At its core, our technique relies on a procedure that takes as input a small sub-set of strings and produces a semantic regular expression that matches all of the strings (except for outliers). While this exact problem is solved by Chen et al. [7], we use an alternate approach that is significantly simpler and suffices for our use case. Our approach consists of two steps: first, we perform *semantic abstraction* on the string to obtain a *masked string* where semantic entities have been replaced by corresponding masks. Second, we use an off-the-shelf regular expression synthesizer to learn regular expressions over these masked strings. In our implementation, we use the regular expression learning procedure from [24] as implemented in the PROSE program synthesis library [19].

EXAMPLE 2. *Consider learning a semantic regular expression for the strings "US-823-JUN" and "Aus-664-PRO" from Figure 2. First, we use semantic abstraction to mask the semantic entities in the strings. In this case, both strings contain countries and can be masked to Country-823-JUN and Country-664-PRO. These strings are passed to an off the shelf regular expression learner, along with the token Country as a built in character class. Given these, the regular expression learner produces the semantic regular expression Country-[0-9]+-(PRO|CAT).*

One way of obtaining the semantic abstraction of a string is to maintain a dictionary of concepts. This has three main drawbacks: some concepts are hard to exhaustively enumerate (like colors), spelling mistakes cannot be repaired without additional fuzzy matching (like "bleu" instead of "blue") and semantic concepts can be contextual ("red" can also refer to a movie).

We propose to leverage a large language model (LLM) to obtain the semantic abstraction. To capture the context of semantic concepts, we prompt the model with a whole column at once. Long columns are processed in batches based on the maximum prompt length (4K tokens for GPT-3.5). Due to the repetitive nature of this task, we found long prompts did not affect the generation quality.

It is important to mask values at the right level of granularity. For example, values from a column ["Q4-2002", "Q3-2002", "Q32001"] are masked entirely as Quarter if given to the LLM without further restrictions. This mask prevents the last value from being repaired ("Q32001" → "Q3-2001"). In order to do so, we only mask a set of predefined semantic categories. Sherlock [12], a semantic type detection system, introduced a method to classify a column as one of 78 popular semantic types, such as Name, Country and Currency. We take the 20 most frequently occurring semantic types, which cover 99.2% of values with a detected semantic type from a sample of 25K data columns from our Excel data.

This approach covers global knowledge about common semantic types, but it will be limiting when the domain is very specific. For example, if an organization has three digit codes, like "ABD" and "REF", that have semantic meaning but will not be captured by DATAVINCI. To alleviate this, we allow the addition of custom semantic types, which can be added using a description and few examples during few-shot prompting. Section 6 describe these in more detail.

We use few-shot prompting to show the model to mask the substring "s" of semantic type t as {t(s)}. We provide 5 fewshot examples which are static for all tasks.

For example, the masked version of "US-123" is "{Country(US)}-123" and "u.k.-392" becomes "{Country(UK)}-392". These are then transformed to " m_1 -123" and " m_1 -392" before learning patterns, and m_1 is added to the alphabet for our regular expression learner.

Using language models for labeling is computationally expensive. To mitigate this, DATAVINCI leverages a few techniques to reduce the token costs: (1) caching allows DATAVINCI to store the values and mask concretizations and (2) early termination allows DATAVINCI to label a subset of the column and continue only if semantic content is found in the sample. Since most columns will not have semantic information, this conserves tokens.

Furthermore, language models are susceptible to hallucinations. This can be partially mitigated via soft checks in the masking and concretization process. DATAVINCI has three checks: (1) the mask is a proper substring of the original value, (2) the concretized value is a valid entry from Sherlock types, and (3) there are no conflicts in the mask and their concretizations across the column.

3.2 Pattern and Outlier Detection

The problem of finding regular expression patterns $r_1, \dots, r_n$ and the outliers in a set of strings is known as the *string profiling problem* [24]. The main challenge in string profiling is to produce patterns that are not too general and not too specific, while maintaining a reasonable outlier rate. For example, the singleton pattern $.^*$ is always a solution to any string profiling problem, as well as the solution where each string is its own pattern.

ConfMiner [3] presents a string profiling technique to balance generality with specificity while being robust to outliers. Our string profiling procedure is an extension of this algorithm to semantic regular expressions, with modifications to make it more efficient and converge quicker in this setting. For making the presentation self-contained, we reproduce the algorithm from [3] here in full and highlight the novel aspects. Intuitively, the procedure uses a combination of (1) a scoring function Score over patterns which gives lower (better) scores to simple, specific patterns; and (2) an outlier penalty $op \in \mathbb{R}$ to control how many strings are allowed to be outliers not matched by any primary pattern. The algorithm aims to produce a string profile $r_1, r_2, \dots, r_n$ to minimize

$$\sum_i \text{Score}(r_i) + op \cdot |\text{outliers}| \quad (1)$$

where outliers is the set of strings not matched by any of r_i , i.e., $\text{outliers} = \{v \in C \mid \forall r_i. v \not\models r_i\}$.

Algorithm 1 shows the pseudo-code for this string profiling procedure. At its core, the algorithm is a generate-and-select procedure where: (a) a large set of candidate regular expressions are generated (lines 1-5), and (b) a set of regular expressions are selected from these candidates to be the primary patterns (lines 6-8). The “*” in conditionals refers to non-deterministic choices.

Candidate pattern generation. Automatically generating accurate semantic regexes describing a column is a challenging task [25, 28, 29]. On the other hand, manually writing these regular expressions is also infeasible—spreadsheets sometimes contain 100s of columns (median 74.3 in our corpus), and further, spreadsheet users are often novices without experience with programs or regular expressions.

The key insight in the generation step is that useful (semantic) regular expressions may be generated using a very small number of *generator* strings. For example, given the strings “QUAL-47” and “QUAL-21”, it is fairly straightforward for an off-the-shelf regular expression learner to produce the semantic pattern $\text{QUAL-}[0-9][0-9]$. Though a column may contain hundreds of strings matching this primary pattern, a small sample of 2-3 strings are sufficient to generate the pattern. Hence, in the first step of Algorithm 1 (lines 1-5), we sample small subsets of strings from the column and use them as generators to learn candidate cluster patterns using the semantic masking followed by regular expression learning as described in Section 3.1. The generation proceeds until a sufficient number of varying clusters are generated—in practice, we stop the cluster generation when no *new* clusters are generated in 10 iterations.

The sampling of generators itself is guided by the semantic entities appearing in the strings of the column. Intuitively, we partition the column into different semantic profiles based on the set of semantic entities appearing in the strings, and for each sampling round, the strings are sampled from the same profile (see Example 3). Additionally, with a low probability (10%), we sometimes pick the full set of generators from a previously produced cluster. This is to account for cases where a previously generated cluster covers proper subset of a target pattern (see Example 4).

Algorithm 1 The LEARNPATTERNSWITHOUTLIERS procedure

Require: Column C of table T

Require: Scoring function $\text{score} : \text{SemanticRegexes} \rightarrow \mathbb{R}$

Require: Outlier penalty $\text{op} \in \mathbb{R}^+$

Ensure: $\langle r_1 \mid r_2 \mid \dots \mid r_k, \text{outliers} \rangle$

```

1: clusters  $\leftarrow \emptyset$ 
2: while * do ▷ Generate candidate clusters
3:   generators  $\leftarrow \text{SemanticEntityGuidedSample}(C, \text{clusters})$ 
4:    $r \leftarrow \text{LearnSemanticRegex}(\text{generators})$ 
5:   clusters  $\leftarrow \text{clusters} \cup \langle \text{generators}, r, \{v \in C \mid v \models r\} \rangle$ 
6: clustersprimary  $\leftarrow \text{ApproxExactSetCover}(\text{clusters}, \text{score}, \text{op})$ 
7: outliers  $\leftarrow \{v \in C \mid \forall (g_i, r_i, C_i) \in \text{clusters}_{\text{primary}}. v \not\models r_i\}$ 
8: return  $\langle r_1 \mid r_2 \dots r_n, \text{outliers} \rangle$ 
9:
10: procedure SEMANTICENTITYGUIDEDSAMPLE( $C, \text{clusters}$ )
11:   semProfile $i$   $\leftarrow$  Set of semantic entities in  $v_i$  for  $v_i \in C$ 
12:   profile  $\leftarrow \text{Random}(\text{semProfile}_i)$ 
13:   Cprofile  $\leftarrow \{v_i \in C \mid \text{semProfile}_i = \text{profile}\}$ 
14:   generators  $\leftarrow \emptyset$ 
15:   while * do ▷ Iterate 2 – 4 times
16:     if * then ▷ Choose generator string randomly
17:       generators  $\leftarrow \text{generators} \cup \{\text{Random}(C_{\text{profile}})\}$ 
18:     else if * then ▷ Add generators from prior cluster
19:        $(g_i, r_i, C_i) \leftarrow \text{Random}(\text{clusters})$ 
20:       generators  $\leftarrow \text{generators} \cup g_i$ 
21:   return generators

```

EXAMPLE 3. In the running example from Figure 2, we have 2 semantic profiles in the column Player ID: (a) the empty set, where no semantic entities are present for strings such as "QUAL-47"; and (b) the singleton set "Country" for string such as "US-823-JUN" and "austria_837". Say we sample a subset of 2 generators. We get 3 cases:

- Both strings are from semantic profile (a), for example, strings like "QUAL-47" and "QUAL-21". Here, the learned pattern is $r_1 = \text{QUAL}-[0-9]^+.$
- Both strings are from semantic profile (b) and are not outliers, for example, strings like "US-823-JUN" and "Ind-674-PRO". Here, the learned pattern is $r_2 = \text{Country}-[0-9]^+-\text{CAT1}.$
- Both strings are from semantic profile (c) and one of them is the outlier "austria_837". Here, the learned pattern is the more complex regular expression $r_3 = \text{Country}[_-][0-9]^+.*$

EXAMPLE 4. Consider a column of dd-mm-yyyy dates and a pattern $[0-9]\{2\}-[0-9]\{2\}-20[0-9]\{2\}$ learned from generators "23-04-2022" and "03-12-2011". Now, this pattern only includes dates starting from the year 2000. Line 20 in Algorithm 1 is aimed at this situation. Rather than having to randomly sample dates from both before and after 2000, the algorithm may instead select the whole set of prior generators along with new generator strings (say "09-06-1999") to obtain the target pattern $[0-9]\{2\}-[0-9]\{2\}-[0-9]\{4\}.$ Note that this step is an optimization to make the learning of patterns faster, i.e., with fewer candidate cluster generations—even without, we will be able to generate the right cluster albeit with more attempts at sampling a diverse enough set of generators.

Primary pattern selection. Once a set of candidate patterns is generated, we select a subset of them as primary patterns. We treat this as a variant of the minimum-cost exact set cover problem: (1) The full set $\mathcal{S}$ is given the set of strings in the column C ; (2) The subsets used to cover the full set are given by $P_i = \{v \in C \mid v \text{ matches candidate pattern } r_i\}$ for each candidate pattern r_i ; and $O_j = \{v\}$ for each string v in C . That is, we are aiming to cover the full set of strings in C using patterns P_i and outliers O_j . *exact* in minimum-cost exact set cover problem refers to the fact that all selected subsets must be disjoint. The cost for selecting each pattern set P_i is $\text{Score}(r_i)$ and for the outlier sets O_i is the outlier penalty op .

An optimal solution for this problem precisely gives the set of primary patterns and outliers that minimizes $\sum_i \text{Score}(r_i) + op \cdot |\text{outliers}|$. However, these problems are NP-hard and we use the classical approximation algorithm for set cover (see, any introductory textbook on algorithms—e.g., [9]). Intuitively, (a) we start with no subsets selected, (b) in each iteration, we select the subset S that minimizes $\frac{\text{Score}(S)}{|S|}$, (c) we eliminate any subset that has a non-empty intersection with the selected subset, and (d) repeat until the union of the selected subsets is equal to the whole set.

EXAMPLE 5. *Continuing from Example 3, the primary patterns are picked from the 3 candidate patterns r_1 , r_2 , and r_3 . approximate exact set cover algorithm, where each candidate pattern is treated as equivalent to the set of strings in the column matched by that pattern. The approximate set cover algorithm picks patterns based on maximizing $\frac{|\{v \in C \mid v \in r_i\}|}{\text{Score}(r_i)}$ —patterns that cover larger subsets of the column are preferred and patterns that have a lower score are preferred. In our case, r_1 and r_2 are in the sweet spot, covering a large fraction of the column while being scored well based on their simplicity. The pattern r_3 are scored very high due to their complexity and not picked. Eventually, the exact set cover algorithm picks the primary patterns r_1 and r_2 , along with the sole outlier set "austria_837".*

As seen in the example above, obtaining good patterns using this algorithm depends heavily on the scoring function and the outlier penalty. We therefore borrow these finely tuned components from [3] and [24], while extending the scoring function to semantic constructs in a straight-forward way. More precisely, we give each semantic entity (e.g., the entities Country or Currency) a constant score. This score is relatively low compared to the standard non-semantic constructs based on the reasoning that semantic entities are very strong constraints and it is unlikely that a substring unintentionally matches a semantic entity. For example, the substring "Vanuatu" is significantly more likely to represent a country Country inside a string rather than an arbitrary combination of letters $[A-Za-z]^*$. For example, Country costs 0 while $[A-Za-z]^*$ costs 6, for "Vanuatu" We did not tune the precise value of this constant score for semantic entities, and instead arbitrarily chose a low value (0). Choosing higher values for this score will lead to cases where substrings that match semantic entities will be matched using non-semantic operations—this will lead to a lower error detection rate.

REMARK 1. *The stochastic sampling based profiling approach is well-suited for the cases where (1) the number of primary patterns is expected to be much smaller than the number of strings in the column, and (2) each primary pattern can be learned from a few positive examples. Intuitively, the algorithm relies on sampling subsets that only contain strings from one primary pattern, while being diverse enough to learn the primary pattern instead of a sub-language of it. As in Example 3, in practice, we see that this hold in common scenarios.*

3.3 Generating Repair Sketches

Given a pattern $r_k \in R_m$ and a value $v \notin \mathcal{L}_{r_k}$, the procedure `GenerateRepairSketches` produces *repair sketches* or templates for a corrected value accepted by r_k . In the next section, we will show how we concretize these repair sketches into repaired strings.

Formally, a *repair sketch* for a pattern r_k and a string value v is a sequence of tokens $t_1 t_2 \dots t_n$ where: (a) each token is either a character c or a hole H_f where f is a fragment of the pattern r_k , (b) the sequence of characters appearing in the $t_1 \dots t_n$ is a sub-sequence of v , and (c) replacing each H_f in the repair sketch with strings v_f that each match the corresponding f results in a string that matches r_k . Intuitively, a repair sketch is obtained from v by deleting select sub-strings and inserting holes.

EXAMPLE 6. Consider a pattern $[0-9]\{2\}-[0-9]\{2\}-[0-9]\{4\}$ and a string $02-11-12$ not matching the pattern. Here, the pattern represents dates with a four digit year, while the string has a two digit year. The sketches $"02-11-H_f 12"$ and $"02-11-12H_f"$ where $f = [0-2]\{2\}$ are both valid repair sketches for the above pattern and string. Similarly, for pattern $\text{Country}-[0-9]+-\{\text{CAT1}\}$ and string $"austria_837"$, $"H_{\text{Country}-837-H_{\text{CAT1}}}"$ is a valid repair sketch.

To produce candidate repair sketches given a pattern r and a string v , DATAVINCI uses a dynamic programming approach to find the set of least cost repair sketches. Here, the cost of a repair sketch is given by the number of delete, substitute, and insert operations needed to transform the string into the repair template. This task of finding the least cost repair sketches corresponds very closely to the problem of approximate regular expression match which has been well studied. We do not discuss these works in detail, but point the reader to the typical approaches [2, 5, 20] and the survey Navarro [22]. The main addition in our approach is that rather than inserting only characters, we also incorporate the insertion of holes corresponding to LLM mask values and unions in a straight-forward manner.

EXAMPLE 7. Continuing from Example 6, the repair sketches $"02-11-H_f 12"$ and $"02-11-12H_f"$ where $f = [0-2]\{2\}$ for the string $"02-11-12"$ both have a distance of 1 from the string corresponding to the insertion of the hole H_f , and are both among the least cost repair sketches. Similarly, for the pattern $\text{Country}-[0-9]+-\{\text{CAT1}\}$ and string $"austria_837"$ from Figure 2, the cost of the repair sketch $"H_{\text{Country}-837-H_{\text{CAT1}}}"$ is 4 based on the operations to substitute $"_"$ with $"-"$ and adding a second $"-"$ and $"\{\text{CAT1}\}"$

3.4 Concretizing Repair Sketches

DATAVINCI concretizes minimum-cost repair sketches produced for each pattern r_k and non-matching string v in Section 3.3 into repaired values. To do so, DATAVINCI fills the holes H_f in repair sketches with strings matching f , using the clean non-outlier rows of the table as examples.

First, we define $v_{r \downarrow f}$ to be the substring of a string v that is matched by f when matching v with a pattern r . In the running example, considering $r = \text{Country}-[0-9]\{3\}-\{\text{CAT1}\}$, $f = \{\text{CAT1}\}$, and $v = \text{"US-823-JUN"}$, the value $v_{r \downarrow f}$ is "JUN" . Note that $v_{r \downarrow f}$ is not necessarily uniquely defined when the match of r with v is ambiguous. For example, for $r = [\text{A-Z0-9}]*[0-9]^*$ and $v = \text{"ABC123"}$, the sub-string of v matched by the fragment $[0-9]^*$ can be one of "123" , "23" , or "3" . In practice, this case of ambiguous matches does not arise often due to the restriction class of regular expressions we consider. When ambiguity does arise, we resolve it by arbitrarily picking one match and using the corresponding $v_{r \downarrow f}$ value.

To concretize repair sketches, we use a divide-and-conquer approach and fill each hole separately. For each row for which the column value v^* matches the pattern r , we take the value $v_{r \downarrow f}^*$ as the expected output and the values in the associated row as inputs. With this set of input-output pairs, we learn a prediction function that predicts the value to be filled into the hole in the repair sketch given the values in the associated row. We handle the task of doing this prediction separately for semantic token holes and non-semantic (syntactic) holes.

For syntactic holes, DATAVINCI fits a decision tree over features extracted from the example inputs, maximizing prediction accuracy. It then applies this decision tree to produce a concrete

substring for the associated hole. We use the feature function presented in CORNET [32], a system for learning conditional formatting rules for tables. The feature function creates boolean predicates from every value in a row tuple by applying the templates described in Table 1. DATAVINCI can then learn a decision tree over such predicates.

Table 1. Supported predicate templates and their arguments. v denotes the column value. For example, `equals(col1, "AR")` matches the cells in column `col1` which are equal to "AR".

<code>equals(v, s)</code>	<code>contains(v, s)</code>	<code>startsWith(v, s)</code>
<code>isLogical(v)</code>	<code>isNum(v)</code>	<code>isError(v)</code>
<code>endsWith(v, s)</code>	<code>length(v, n)</code>	<code>hasDigits(v)</code>
<code>isNA(v)</code>	<code>isFormula(v)</code>	<code>isText(v)</code>

EXAMPLE 8. In Figure 2, given the outlier value "`usa_837`", DATAVINCI produces the repair sketch `Country-837-(JUN|PRO)` with two holes, the latter of which is resolved by our decision tree approach. DATAVINCI takes non-outlier row tuples and their concrete hole value

`Professional` `Ind-674-PRO` $\rightarrow$ `PRO`
`Junior` `US-823-JUN` $\rightarrow$ `JUN`
 $\dots$

as examples. After applying the CORNET feature functions, we will learn a single node tree that predicts "PRO" if Category is equal to "Professional" and "JUN" otherwise. This tree will thus concretize the second hole in our sketch to be "PRO".

For semantic token holes, DATAVINCI uses an LLM to produce a prediction. To do so, we use a prompt where the examples from the non-outlier rows as described above are given as few-shot examples. Additionally, the incorrect value v is also provided in the prompt. In certain cases, DATAVINCI optimizes the masking call to the LLM by extending the task to also generate the concrete value. For example, in Figure 2, the LLM generated masked output for the outlier value is "`Country(AUT)_837`" which contains the country mask and also the LLM predicted concrete value ("`AUT`").

For both types of hole resolutions, DATAVINCI aims to reduce spurious edits by relying on the confidence of the associated predictor. For example, given two repair sketches for phone numbers of the form "`21H(0-9)-531-4428`" and "`215-531-442H(0-9)`", where the hole $H_{(0-9)}$ must be resolved in both, the first is likely to meet our confidence threshold α (default to 0.9) if enough non-outlier values share an area code, while the second is unlikely to result in a suggestion as ending phone digits do not typically reflect information. Similarly, for an LLM hole in a sketch such as "`HCountry-Paris`" is likely to result in France as a prediction, given low-temperature sampling of the LLM's predictions.

3.5 Ranking Repair Candidates

Our repair procedure can produce multiple edit programs (since there may be multiple significant patterns). To address this challenge, DATAVINCI uses a heuristic candidate ranker. This heuristic corresponds to a weighted linear combination of edit script properties. The weights are manually set based on qualitative analysis on a small held-out set of 100 columns sampled from our corpus of Excel spreadsheets. The four properties are (1) string edit distance between erroneous value and the repaired value, (2) count of alphanumeric edit operations, (3) string edit distance of repaired value to closest value in column, and (4) fraction of column matching the significant pattern used to generate the repair.

The weights are heuristically set based on a held-out set of 100 columns sampled from our corpus of Excel spreadsheets. We list examples to motivate each of the components used in ranking.

EXAMPLE 9. Consider, the repair candidates shown in Figure 2 (5). We can see that "AUT-837-PRO" has a repair edit distance of 5 while "QUAL-837" has a repair edit distance of 8.

EXAMPLE 10. DATAVINCI's ranker also prefers non-alphanumeric edits over alphanumeric. Consider the outlier string, "IND12" and the repair candidate "IND-12" versus "IND1". Both of these have an edit distance of 1 but DATAVINCI ranker prefers the first one.

EXAMPLE 11. Consider a column of dates (DD/MM/YY) all in the 18th century, and the outlier "12/04/1992" with two possible repair candidates ["12/04/19", "12/04/92"]. Since the edit distance to the closest value is lower for the second candidate and hence, DATAVINCI ranker picks the correct candidate.

4 Evaluation Setup

We describe the benchmarks and baselines over which we evaluate.²

4.1 Benchmarks

We use a benchmark of web tables from prior work [11] and we collect and release three new Excel-based benchmarks. We briefly describe the statistics of these benchmarks in Table 2.

Wikipedia Tables. We build on the Wikipedia tables dataset released in the original Auto-Detect [11] paper. Following their approach, we take the sample of 1000 tables on which they manually annotated system predictions and extend this with manual annotations for DATAVINCI and our baselines. Like prior work, given this annotation approach, we only report precision [11].

Excel. We sample 200 tables present in workbooks drawn from a corpus of 1.8 million publicly available Excel workbooks [32]. Similar to the Wikipedia benchmark, we run all available systems on the tables, manually annotate their suggestions, and report precision.

Synthetic Errors. We sample 1000 Excel tables (disjoint from the Excel benchmark) from the same corpus previously described. We then synthetically introduce errors with the goal of measuring recall. We introduce both syntactic and semantic errors. To introduce syntactic errors, we apply the following noise operations: (1) random character insertion, deletion and change, (2) random delimiter insertion deletion and change, (3) random digit swap, (4) random shuffle of characters, (5) random capitalization, (6) random decimal, and (7) comma swap in numbers. Similarly, for semantic errors we apply: (1) spelling errors, (2) visually-inspired typos $\{o \rightarrow 0, l \rightarrow 1, e \rightarrow 3, a \rightarrow 4, t \rightarrow 7, s \rightarrow 5\}$, (3) shuffling values in a column, (4) entity swap $\{\text{"India"} \rightarrow \text{"Japan"}\}$. We randomly corrupt cells with 20% probability. For each of the cells to be corrupted, there is a 25% probability of applying 1, 2, 3 or 4 noise operations, sampled without replacement from the set of operations described. Because it is likely there are already real data errors present in the data, systems may detect errors or suggest repairs for cells beyond our synthetically corrupted cells. As a result, we focus our analysis on recall of our synthetic errors, but report precision (naturally deflated) and F1 for completeness.

Excel Formulas. We create a dataset of the form (formula, input columns) where formula is an Excel formula used to create a new column (all rows have the same formula) from the input columns. We restrict to formulas where input and output value are in the same table. The task is to repair the input columns such that the formula executes without errors. We sampled 15,000 tables from Excel corpus previously described, extracted 11,000 formulas where at least 1 cell and less

²Data will be released for the camera ready to undergo required compliance checks.

Table 2. Benchmark properties and metrics reported. # Rows/# Cols denote the mean rows/columns.

Dataset	Metrics	# Tables	# Columns	# Rows
Wikipedia Tables	Precision, Fire Rate	1000	5.1	27.3
Excel	Precision, Fire Rate	200	1.6	523.4
Synthetic Errors	Precision, Recall, F1 Score	1000	4.3	447.5
Excel Formulas	Execution Success Rate	11000	1.4	216.5

then 25% of cells result in an error value. Of these 11,000 formulas, 7,200 have a single column input and 3,800 have multiple column inputs (on average 3.4). We use this dataset to evaluate the impact of execution-guidance.

4.2 Baselines

We compare against various baselines:

- (1) **WMRR** [1] is an unsupervised approach to learn weighted data rectifying rules based on functional dependencies. Since the tool is not publicly available, to evaluate against WMRR we re-implement it based on their paper description.
- (2) **HoloClean** [31] is a popular data repair tool based on probabilistic inference, which can repair qualitative and statistical errors. HoloClean uses a bag of outlier-detection techniques, some of which are based on denial constraints and external datasets and some of which are based on unsupervised outlier detection. We run the code released by the authors on GitHub. HoloClean originally requires that users provide denial constraints. To evaluate in a fully unsupervised setting, comparable to DATAVINCI, we use a single vacuous denial constraint (specifically, column 1 = column 1).
- (3) **Raha** [17] is an error detection system that is based on an ensemble-like architecture that uses semi-supervision to train an error detection system. As Raha requires annotated examples, in our evaluation we consider three variants for providing examples: (1) the first (top-to-bottom) 5 ground truth errors per column, (2) 10% of total errors in the column, and (3) the suggested example count (20) by the authors.
- (4) **Auto-Detect** [11] is a co-occurrence-based error detection system, which also uses regular-expressions to generalize values. We use the Wikipedia results released with the paper. Unfortunately, we are unable to run the tool on all our benchmarks as it is not available publicly and as a result only report performance on Wikipedia.
- (5) **Uni-Detect** [36] is a statistical method that tests a null hypothesis of a table being *normal* (no error) based on the alternative hypothesis that some rows need to be removed for the table to become normal. The *normality* is computed using features defined for different types of errors, like standard deviation for numerical outliers and minimal edit distance for spelling errors, over a corpus of 100M tables.
- (6) **Potter's Wheel** [30]: a seminal data error detection and (semi-supervised) correction system based on functional dependencies. We leverage the Potters Wheel's error detection annotations on the Wikipedia dataset released in the original Auto-Detect paper. Since we rely on public annotations, we only report Potter's Wheel on the Wikipedia benchmark.
- (7) **T5** [27], **Phi-2** [15], **Mistral** [13], **Llama-2** [33] are popular transformer-based encoder-decoder model pre-trained on text. We fine-tune these for the task of data repair. Since these are text generation model we encode each column as a stringified list of column values separated by a [SEP] token. The model is trained end to end to generate the repaired column, given the potentially noisy column as input. The training data consists of 100K dirty samples

(generated by the same approach used in our synthetic benchmarks) and we task it with generating the original columns. Because we run these on a single column at a time, it does not consider other columns while repairing.

- (8) **GPT-3.5** [4] and **GPT-4** [23] are state-of-the-art transformer-based decoder-only model. We use the same input structure used to train T5 to include the target column in GPT-3.5 and GPT-4's prompt. We use these in a few-shot setting, providing three static examples of a dirty column and the cleaned output the model needs to generate. The static examples are from Excel (disjoint from our benchmarks) and are hand annotated. We report results at temperature 0 and top-1 generation. After experimenting with multiple temperatures, we found temperature 0 works best on average based on precision and F1 score across all benchmarks.

To evaluate unsupervised repair when using detection-only systems, such as Raha, Auto-Detect and Potter's Wheel, we add a call to GPT-3.5 where we include the outlier value and its column header along with 10 sample values selected based on spatial proximity (5 rows above and below and 3 columns to the left and to the right with headers). We ask the model to generate the repaired value. We sample values to fit in the fixed prompt length of 4,000 tokens and make individual repair calls for each outlier detected.

4.3 Metrics

To measure the quality of detection and repair we report three metrics: **precision**, **relative recall**, and **fire rate**. Precision is the standard precision over the model outlier detection and repair. Relative recall is defined as the recall over the set of prediction done by DATAVINCI and all other baseline systems and is commonly used in sparse annotation domains like data cleaning [11]. Fire rate is simply the percentage of tasks where a prediction was made in the form of an outlier being detected or repaired.

5 Results and Discussion

We perform experiments to answer the following research questions

- RQ1.** Can DATAVINCI accurately detect string errors?
- RQ2.** Can DATAVINCI accurately repair string errors?
- RQ3.** How do DATAVINCI's design decisions impact performance?
- RQ4.** What are system requirements for DATAVINCI?

We do a case study (**CS1**) on program execution to guide repair.

5.1 Error Detection (RQ1)

Like prior work [11, 17, 31] we report precision for detection. We leverage existing annotations where possible and otherwise manually annotate systems' predictions. We report precision, recall, and F1 on our synthetically corrupted dataset, computed with respect to our corruptions, as a result precision/F1 score can be deflated from pre-existing data errors.

In addition to standard metrics like precision, we also report each system's *average fire rate*. We define this as the average fraction of cells in a column that are labeled as data errors. Further, since getting absolute recall requires exhaustive annotation of values which is difficult in our domain due to the low error rate, we report relative recall which we define as recall over the collective set of predictions by DATAVINCI and all other baselines.

Table 3 presents error detection results for DATAVINCI and baselines across three benchmarks. We find that DATAVINCI outperforms baselines in terms of precision on both the Wikipedia and

Table 3. Error detection performance across datasets. DATAVINCI outperforms in terms of precision (Prec.) and relative recall (Re. Recall) on Wikipedia and Excel, and in terms of precision (Prec), recall and F1 on our synthetic benchmark. We report Potter’s Wheel and Auto-Detect results using the annotations released with the Auto-Detect paper. For synthetic benchmarks the ground truth is taken as the original table can skew the precision and F1 score as explained in Section 4, hence these are reported with a (*).

System	Wikipedia			Excel			Synthetic		
	Prec	Re. Recall	F.R.	Prec	Re. Recall	F.R.	Prec *	Recall	F1 *
WMRR	70.0	11.4	2.93%	65.8	12.5	2.76%	55.3	66.8	60.5
HoloClean	67.0	14.4	3.87%	65.2	11.8	2.50%	52.1	64.1	57.5
Raha	68.9	15.4	4.03%	66.4	13.7	3.74%	59.5	68.2	63.6
Raha (Best)	76.5	23.6	9.12%	68.1	19.2	7.81%	64.2	73.4	66.8
Potters-Wheel	66.2	-	-	-	-	-	-	-	-
Auto-Detect	78.5	-	-	-	-	-	-	-	-
Uni-Detect	74.1	67.3	18.34%	63.3	70.2	16.38%	62.1	63.0	62.5
T5 – (Trained)	60.8	92.6	27.47%	53.8	90.4	19.02%	40.5	56.3	47.1
Phi-2 – (Trained)	63.4	81.6	23.21%	55.2	86.4	18.11%	42.3	58.1	49.3
Llama2 – (Trained)	64.1	77.6	21.85%	56.1	86.1	17.55%	43.1	57.0	47.8
Mistral – (Trained)	63.1	80.5	25.42%	54.9	87.1	18.77%	41.8	56.8	48.7
GPT-3.5	73.9	56.3	10.99%	60.4	64.4	11.71%	50.1	69.8	58.3
GPT-4	74.1	41.7	10.82%	60.9	62.3	11.23%	50.6	70.3	58.9
DATAVINCI	80.1	93.4	16.85%	75.1	94.7	14.39%	67.4	73.4	70.3

the Excel benchmarks, despite having a higher firing rate than all but one baseline (T5). Auto-Detect, which is well-suited to the type of mistakes present in the Wikipedia dataset, performs competitively. Overall, we find that error detection is relatively easier on the Wikipedia benchmark, where tables on average have fewer rows, compared to the Excel benchmark.

When performing qualitative inspection of the errors detected, we find that GPT-3.5 can identify errors in semantic substrings well. For example, in the following column of financial quarters, {"Q1-22", "Q4-21", "Q5-20", "Q2-20", "Q1-21"} GPT-3.5 correctly identifies the outlier to be "Q5-20". However, GPT-3.5 fails to detect syntactic errors like "S1.4" in the column, {"S1.2", "S2.3", "S1.4", "S1.3", "S2.1"}, where "S1.4" is missing a period after "S". Neural models like GPT-3.5 struggle at recognizing these patterns whereas DATAVINCI can detect such errors using regular-expression-based patterns.

Other tools, like Auto-Detect, work well on syntactic errors but fails on semantic repairs. For example, consider the column of county and a numeric ID separated by a hyphen {"Alpine_231", "Kings_721", "Lake_201", "Santa Clara_246", "Nevad210"} the correct repair here is "Nevad210" → "Nevada_210". This error involves a combination of syntactic and semantic inconsistency which most baseline systems struggle with. DATAVINCI combines semantic information via masking into its pattern based syntactic repair engine and thus detects this error (and generates the correct repair).

5.2 Error Repair (RQ2)

For repair, we find that often there are cases where various possible data repairs are reasonable and the correct repair cannot be uniquely identified. To account for this, we annotate repair suggestions

Table 4. Error repair performance across datasets. As described in Section 4 for the Wikipedia and Excel benchmarks we report repair precision split into (1) Certain: repairs that are certain (based on hand annotation) and, (2) Possible: repairs that are reasonable but groundtruth cannot be uniquely determined. For synthetic benchmarks the groundtruth is taken as the original table which can also have inherent data errors which will skew the precision and F1 score hence, these metrics are reported with a (*). Recall is computed as the percentage of cases correctly repaired out of the total errors synthetically introduced. Finetuned systems are indicated with (*Trained*). Potter’s Wheel is denoted as PWheel in the table.

System	Wikipedia		Excel		Synthetic		
	Precision (Certain)	Precision (Possible)	Precision (Certain)	Precision (Possible)	Precision *	Recall	F1 *
WMRR	61.1	57.8	59.2	55.6	43.2	61.1	50.6
HoloClean	58.4	55.6	59.0	54.9	41.3	58.6	48.5
Raha + GPT-3.5	58.6	54.8	56.4	53.5	45.2	62.0	52.3
Raha (Best) + GPT-3.5	64.3	57.2	59.0	56.2	48.1	65.2	56.5
PWheel+GPT-3.5	56.2	52.0	–	–	–	–	–
AutoDetect + GPT-3.5	66.9	63.3	–	–	–	–	–
UniDetect + GPT-3.5	67.3	63.6	57.2	54.0	44.3	60.5	51.3
T5 – (<i>Trained</i>)	41.0	37.8	37.7	35.2	27.9	47.0	35.0
Phi-2 – (<i>Trained</i>)	41.7	38.3	38.1	35.5	28.4	46.3	34.8
Llama-2 – (<i>Trained</i>)	42.5	39.4	38.6	35.3	28.8	46.4	35.0
Mistral – (<i>Trained</i>)	41.5	37.9	38.0	35.3	28.2	46.8	34.9
GPT-3.5	63.9	55.5	52.1	48.9	38.2	63.8	47.8
GPT-4	64.2	55.6	52.2	49.3	38.5	64.2	48.2
DATAVINCI	71.3	64.9	71.2	64.6	54.1	68.9	60.6

as *possible* if this is the case, and annotate the suggestion as correct or not. In our results, we report repair precision for certain and *possible* cases separately for completeness.

Table 4 shows the performance of DATAVINCI and baselines systems for repairing data on our benchmarks. Note that repair metrics combine: (1) detection (as a system must identify an error to fix it), and (2) whether the repair matches the ground-truth repair.

We find that DATAVINCI outperforms all baselines in terms of both certain and possible repairs on Wikipedia and Excel benchmarks and has the highest precision, recall and F1 score on the synthetic test set. Raha+GPT-3.5 and Auto-Detect+GPT-3.5 have high precision (Wikipedia), but we find that they have different behaviors. Specifically, Auto-Detect (by design) does not support inter-column dependencies, while Raha struggles to detect intra-column patterns. WMRR and Potter’s-Wheel capture both inter- and intra-column dependencies well but struggle with semantic repairs as they do not detect these issues. Both GPT-3.5 and T5 perform significantly worse on the synthetic dataset as these have a mix of semantic and syntactic errors.

Table 5 shows the precision rates when we only consider correctly detected errors for each system as a way to disentangle the detection and correction effectiveness of each system. We find that repair precision is substantially higher across the board, if we only consider correct detections, and DATAVINCI performs the best.

Table 6 shows the breakdown of the repair precision rates over the synthetic dataset. We show performance broken by the type of synthetic error (syntactic, semantic, or mixed). We find that neural models (T5, GPT-3.5) have high performance on semantic but struggle with syntactic errors. Rule based systems (HoloClean, WMRR) have high performance on syntactic but struggle with

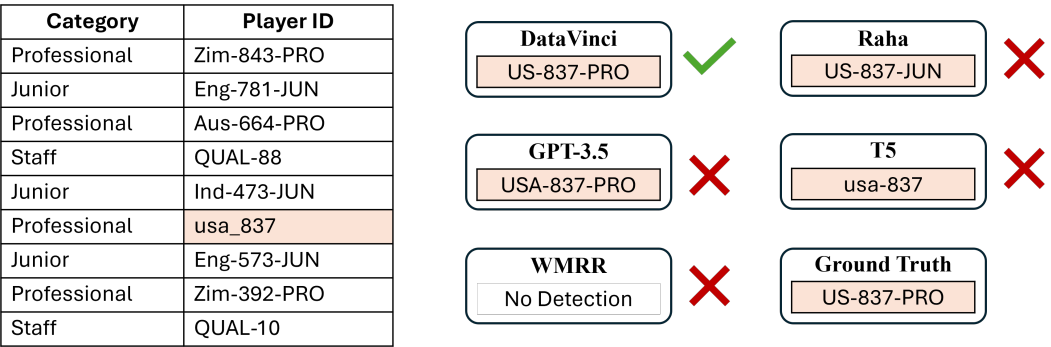


Fig. 3. Example from Excel benchmarks where DATAVINCI generates the correct repair while baseline systems fail. The ground truth is US-837-PRO. DATAVINCI combines semantic and syntactic substrings in its pattern and repairs the column correctly. Analysis for this example is presented in Figure 2.

Table 5. Table showing repair precision as the percentage of errors that DATAVINCI and baseline systems can repair correctly out of the errors that were correctly detected by each system. DATAVINCI has the highest repair rate.

System	Wiki	Excel	Synthetic
WMRR	87.3	89.9	78.2
HoloClean	87.1	90.5	79.3
Raha + GPT-3.5	85.0	85.0	76.0
Raha (Best) + GPT-3.5	89.2	87.5	79.2
Potters-Wheel + GPT-3.5	84.9	–	–
Auto-Detect + GPT-3.5	85.2	–	–
Uni-Detect + GPT-3.5	83.1	84.0	77.4
T5	67.4	70.1	68.8
Llama-2	68.3	70.8	69.9
GPT-3.5	86.5	86.3	76.3
GPT-4	86.9	86.7	76.9
DATAVINCI	89.0	91.2	80.3

semantic errors. Combining neural and rule based systems (Raha + GPT-3.5) can achieve high performance on both semantic and syntactic errors individually but struggles with mixed syntactic and semantic errors. DATAVINCI outperforms baselines across all error type with the highest differential in mixed errors (+5%).

We look at cases where DATAVINCI is able to accurately repair data errors while baselines fail. We find that these are mostly where either (1) DATAVINCI is able to leverage its semantic masking to suppress false positives; or (2) DATAVINCI detects a semantic anomaly using patterns. Figure 3 shows an example from the Excel benchmarks, which contains a tournament table having columns “Category” (“Junior” or “Professional”) and “Player-ID” which has three components (Country code, unique numeric ID, first three letters of category). For non competing players, the ID is “QUAL-” followed by a unique numeric ID. “usa_837” is an outlier in the Player-ID column and the correct repair should change it to “US-837-PRO”. As highlighted above, DATAVINCI utilizes (1) semantic

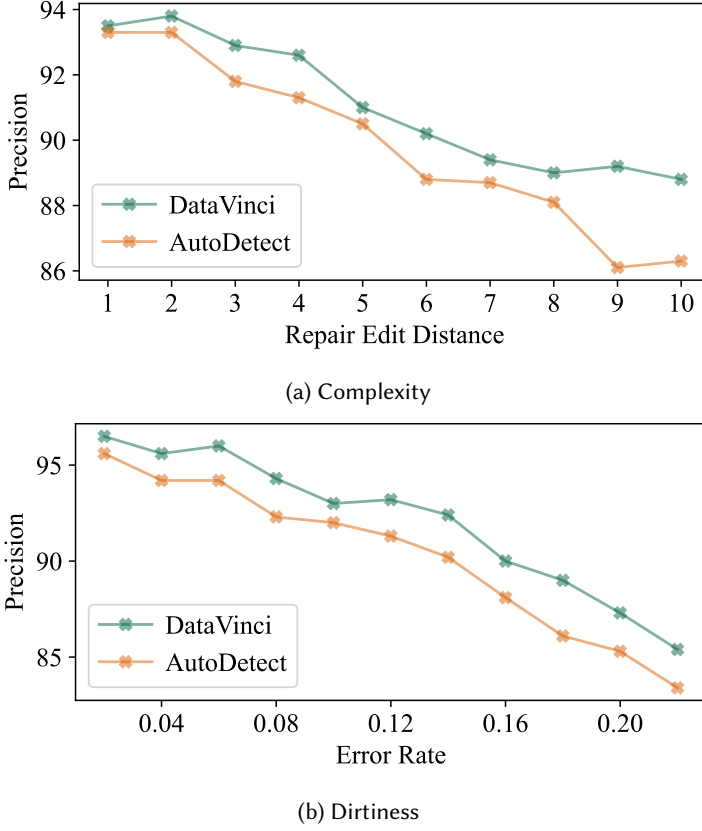


Fig. 4. Plot showing repair precision for DATAVINCI and best baseline against increasing (a) repair complexity as denoted by repair edit distance and (b) dirtiness of the column as denoted by the error rate of the columns.

masking to repair "usa" $\rightarrow$ "US" and uses (2) patterns, paired with concretization value constraints, to detect that the category substring is "PRO".

We also look at cases where DATAVINCI failed to generate the correct repair but one of the baseline systems succeeded. We find that these cases were mostly the result of either (1) the column does not have any significant patterns due to irregular data, or (2) the error rate is too high and as a result the outlier is covered by a significant pattern. Figure 5 shows one example for each case from the Excel benchmarks along with the incorrect repair generated by DATAVINCI and the correct repair generated by the baseline. In the first example, DATAVINCI learns two significant patterns $R_M = \{[A-Z]^+, [A-Z]^+0\}$ and hence, does not detect the error. In the second example, since the column contains irregular data, DATAVINCI is unable to learn a significant pattern $R_M = \{\}$ and does not detect any errors.

5.3 Design Decision (RQ3)

We investigate the importance of semantic abstraction/concretization, concretization value constraints, and ranking. Table 7 reports results on our synthetically corrupted benchmark.

5.3.1 Semantic Substrings. To evaluate the impact of having semantic information in repairs (see Section 3.1), we evaluate two versions of DATAVINCI. We implement a version of DATAVINCI

Table 6. Breakdown of synthetic error based on type: Syntactic (Synt), Semantic (Sem) and Syntactic + Semantic (Mixed).

Model	Synthetic	Semantic	Mixed
WMRR	81.5	80.4	70.3
HoloClean	81.8	82.6	69.3
Raha + GPT-3.5	78.5	81.4	68.1
T5	71.2	75.6	59.6
Llama-2	72.1	76.2	60.4
GPT-3.5	78.9	82.1	67.9
GPT-4	79.4	82.6	68.4
DATAVINCI	82.1	86.5	75.3

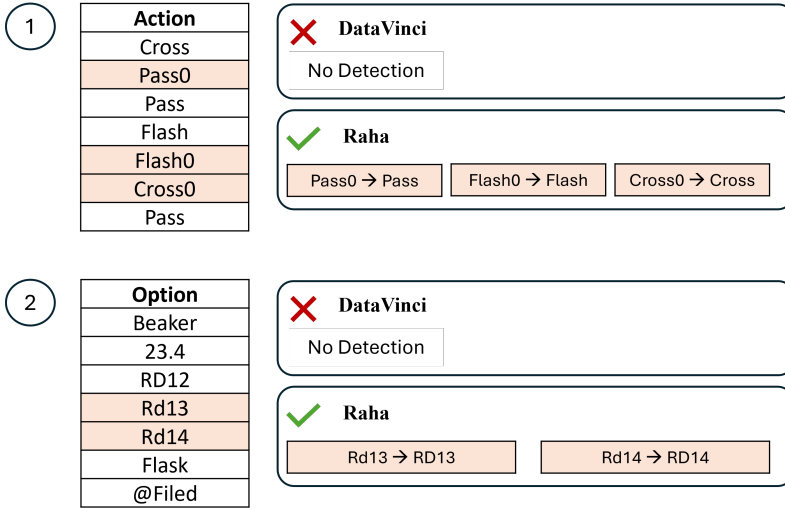


Fig. 5. Sample where DATAVINCI fails to generate the correct repair, but a baseline system succeeds. We show all suggested repairs (denoted by $\rightarrow$). DATAVINCI is unable to detect these errors as they either ① satisfy a significant pattern, or ② the column has no significant pattern with irregular data.

that treats all strings as purely syntactic (*no semantic abstraction*) and a version that can perform semantic abstraction but is restricted to re-use the same substring for concretization, meaning it can not repair errors in semantic substrings (*limited semantic concretization*). We find that both versions result in a lower performance compared to full DATAVINCI, but removing semantics altogether has a comparatively larger impact on precision and F1.

5.3.2 Concretization and Ranking. To study the effect of learned concretization value constraints and ranking (see Section 3.4 and 3.5), we design two ablated version of DATAVINCI, (1) where DATAVINCI does not learn to concretize abstract edits, instead enumerating all candidates and directly passing them to the heuristic ranker (*No learned concretization*); and (2) a version where all candidates are ranked based on the shortest string edit distance with respect to the original data error (*Edit distance ranking*). Table 7 shows that while removing either learned concretization or ranking has a negative impact on performance, learned concretization has a larger effect.

Table 7. Repair precision, recall and F1 score on our synthetically corrupted benchmark for different DATAVINCI ablations. Full DATAVINCI outperforms all ablations. Removing semantic abstraction has the most impact on F1 score.

Model	Precision	Recall	F1 Score
No semantic abstraction	50.3	62.9	55.9
Limited semantic concretization	52.0	65.6	58.0
No learned concretization	46.3	51.0	48.5
Edit distance ranking	53.2	67.1	69.3
DATAVINCI w/ Llama-2	51.6	61.6	56.1
DATAVINCI w/ Phi-2	51.2	62.5	56.2
DATAVINCI	54.1	68.9	60.6

Table 8. Time (ms), disk space (MB), and GPU plus CPU memory used (MB), averaged over (a) Wikipedia and (b) Excel benchmarks. (*) denotes systems which we did not run and reported the stats published by Auto-Detect authors. For (DATAVINCI/Raha/Potter’s Wheel/Auto-Detect) runtime with network latency is in parenthesis. (†) denotes fine-tuned.

(a) Wikipedia

System	Time(ms)	Disk(MB)	Memory(MB)
WMRR	247.4	4.6	914.5
HoloClean	1049.3	996.3	1647.2
Raha	321.8 (2171.8)	65.3	645.4
Potter’s-Wheel*	110.0 (1810.4)	-	-
Auto-Detect*	290.0 (2094.0)	-	-
T5 [†]	858.3	886.2	1534.2
Mistral [†]	1639.5	15859.3	8532.3
Phi-2 [†]	1158.3	5432.4	4313.3
Llama-2 [†]	2311.7	27394.8	8923.5
DATAVINCI	261.5 (2431.4)	5.6	10.5

(b) Excel

System	Time(ms)	Disk(MB)	Memory(MB)
WMRR	281.3	7.5	1034.2
HoloClean	1432.5	1012.4	1734.5
Mistral [†]	1912.4	16103.3	8587.0
DATAVINCI	291.7 (2742.4)	6.3	13.0

5.4 Runtime Performance (RQ4)

Table 8 shows the average time taken, disk space and RAM + GPU VRAM used to detect (Raha, Potter’s Wheel, Auto-Detect) or detect+repair (WMRR, HoloClean, T5, GPT-3.5, GPT-4, DATAVINCI) errors on the Wikipedia benchmark. We report Potter’s Wheel and Auto-Detect based on [11] which reports these metrics in a similar environment and on the same benchmarks. We find that in terms of time and disk space, DATAVINCI is competitive with alternatives such as WMRR, Raha, and Auto-Detect, while using substantially less RAM. We only report inference resources but it is

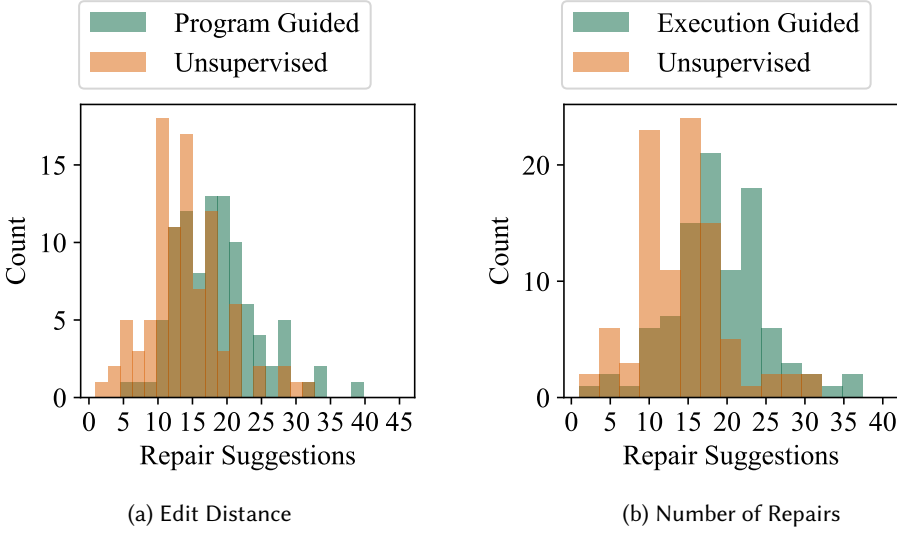


Fig. 6. (a) Distribution of original value to suggested repair edit distances and (b) Number of repairs per column move higher when given execution guidance. This suggests that with execution DATAVINCI can offer more repairs, with higher complexity (as proxied by string edit distance).

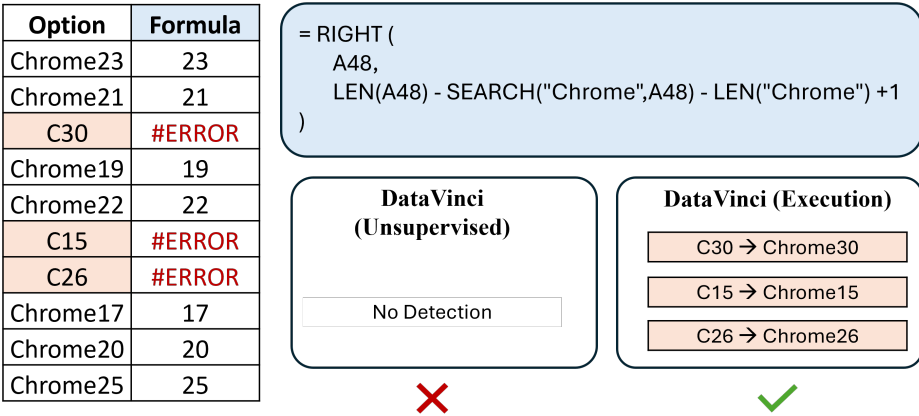


Fig. 7. Sample where DATAVINCI with execution provides repairs leading to successful execution but the unsupervised variant cannot. Since the outlier pattern, $C[0-9]\{2\}$ occurs frequent enough to be considered a significant pattern.

worth noting that models like T5 requires training (4-10 hours on a K80 GPU). Since we use API for language models, we can only report network latency. For DATAVINCI, Raha, Potter's Wheel and AutoDetect, we report both latencies with and without network latency.

5.5 Execution-Guidance (CS1)

DATAVINCI's pattern-based detection relies on the assumption that the significant patterns characterize the data distribution well, and that values that do not satisfy such patterns are data errors.

However, not all string data will be able to produce a significant patterns nor will significant patterns learned over all values necessarily highlight errors. To address this challenge, DATAVINCI can exploit execution information from column transformation programs (which operate on columns and generates a value for each row individually) to further refine its error detection and repair suggestions. This improvement comes from learning a significant pattern set that accounts for different execution outcomes.

DATAVINCI executes the column-transformation program on T and groups executions into successes and failures (as signaled by exceptional values, such as NaN, or program exceptions). The non-exception group is then provided to our regular expression learner and all patterns learned are treated as significant patterns R_m . All values v in our target column that were inputs to a failing execution are identified as data errors, and we apply the repair procedure. Execution information allows DATAVINCI to perform targeted repairs for repairing the data such that the program succeeds.

EXAMPLE 12. *Consider the table in Figure 7. Without execution supervision, DATAVINCI and baselines are unable to generate the correct repair since the outlier pattern $(C[0-9]2)$ occurs too frequently to be categorized as outliers. With execution information, DATAVINCI considers $(C[0-9]2)$ as an outlier pattern and is thus able to generate the correct repair and the program executes successfully.*

We use our Excel Formula benchmark to evaluate the extent to which DATAVINCI can use execution information to provide improved repairs. We report two metrics: the fraction of cells that no longer result in an error value, as well as the fraction of columns where no cell results in an error value (the formula fully succeeds).

To carry out this experiment, we run all baselines, apply repair suggestions *only* on values that are an input into a row that has an error value when the formula is originally executed. We report formula-level and cell-level successful execution rates after applying each systems' suggestions. Table 9 summarizes our results. We find that DATAVINCI with execution-guided learning improves over all baselines³ (including the fully unsupervised DATAVINCI).

Figure 6 shows that the distribution of string edit distances from original data error to repaired suggestion, as well as the number of repairs per column. We find that when provided with execution information DATAVINCI can produce more repairs, and these repairs tend to have a higher distance to the original value (implying possibly more complex repairs). Figure 7 shows an example where DATAVINCI with execution learns the correct repair resulting in successful execution of the formula while the unsupervised variant does not provide any suggestions as the pattern $C[0-9]\{2\}$ repeats enough times to be considered as a significant pattern.

6 Discussion

DATAVINCI relies on language models for semantic masking and concretization. However, using language models for such a task has a few limitations: (1) these models are computationally expensive and slow making their application on longer columns challenging, (2) these models are susceptible to hallucination which is often hard to detect and makes verifying repairs harder, (3) the scope of semantic errors covered by DATAVINCI is bound by the language model's knowledge of the types and this usually starts to break with domain specific repairs. These limitations can be mitigated using strategies explained in the paper, but the general problem remains. For example, in the value "AUT1-340", the language model masks "AUT1" instead of "AUT". DATAVINCI only handles string based repairs which account for 67.6% of table columns, the remaining numeric and date based columns are out of scope for pattern-based techniques and require statistical analysis.

³we exclude HoloClean as it did not scale to this task. We let it run for 24 hours and it only covered 24% of the benchmarks.

Table 9. Formula and cell level execution success on the formulas benchmark. We report the *No Repair* results for comparison. We do not report HoloClean as it did not complete in 24 hours. DATAVINCI with execution performs the best.

Type	Single Column		Multi Column	
	Formula	Cell	Formula	Cell
No Repair	0.0%	85.8%	0.0%	81.4%
WMRR	32.6%	94.4%	29.6%	90.1%
Raha + GPT-3.5	34.5%	92.6%	31.4%	88.3%
T5	11.2%	89.4%	6.4%	86.2%
Mistral	11.2%	89.4%	6.4%	86.2%
Phi-2	11.2%	89.4%	6.4%	86.2%
Llama-2	11.2%	89.4%	6.4%	86.2%
DATAVINCI Unsupervised	43.2%	94.3%	35.7%	90.9%
DATAVINCI+Execution	54.0%	96.5%	47.8%	94.0%

7 Related Work

Data error detection has been the subject of active investigation in the data management community. Prior work has developed systems that can identify qualitative data errors (like incorrect value structure) or quantitative data errors (like unlikely values given a distribution). Potter’s Wheel [30] is an interactive data cleaning system where users annotate data errors with corrections, which the system then learns to apply throughout the data. Recent work like AutoDetect [11] use large-scale co-occurrence statistics with pattern-based generalization to detect errors. Raha [18] detects errors by combining multiple pattern-based error strategies, and gathering user feedback on possible errors from these. Commercial platforms (like Trifacta and PowerBI) typically offer data cleaning based on a fixed set of patterns or common data issues (like leading or trailing spaces). DATAVINCI does not use fixed patterns but rather learns them, does not only detect but also repairs data errors, and handles values with both syntactic and semantic substrings.

While detecting data errors helps identify issues in data, correcting these errors is still costly. As a result, past work has explored automatically repairing identified errors. HoloClean [31] allows users to (optionally) specify denial constraints, which it combines with error detectors, to build a probabilistic data cleaning program. WMRR [1] presents an unsupervised approach to learning cleaning rules, removing the need for users to specify constraints or resolution rules. ActiveClean [14] and Baran [16, 17] are semi-supervised tools to repair data based on few user examples. With the exception of WMRR, this line of work requires a human in the loop.

DATAVINCI can repair strings that contain both syntactic and semantic substrings, as described in Section 3.1. FlashGPT [35] showed that LLM-based semantic transformations can be integrated into a programming-by-example synthesizer that learns syntactic string transformations. Potter’s Wheel [30] anticipated the combination of semantic and syntactic substrings for data cleaning, and allowed users to define semantic rules to guide its transformations.

To support syntactic and semantic repairs, DATAVINCI uses semantic regular expressions, which were first introduced as *semantic string profiles* with FlashGPT [35] and more completely worked out in SMORE [7]. Semantic regular expressions generalize traditional regular expressions to include semantic substrings. SMORE supports both subtyping and logical predicates over semantic entities in their expressions. While DATAVINCI also uses semantic regular expressions, there are multiple key differences: SMORE assumes exact examples and DATAVINCI supports outliers, we simplified

learning by introducing masks, we do not require subtyping, and we are designed for repair instead of extraction.

Transformer models [34] have recently gained a lot of popularity and language [27] and code tasks [37]. As part of our evaluation, we use T5 and GPT-3.5 as baselines to perform data error detection and repair. Prior work [21] has explored using foundation models for data management tasks like validation and cleaning.

8 Conclusion

In this paper we propose DATAVINCI, a tool for automatic repair of string data errors. DATAVINCI learns majority patterns over columns and uses these to detect string data errors. Because errors may occur in strings with both syntactic and semantic substrings, DATAVINCI employs an LLM to mask semantic substrings before learning a pattern. DATAVINCI suggests repairs for the detected errors by deriving minimal edits that lead to satisfying a majority pattern. Since majority patterns may not always occur or capture errors, DATAVINCI addresses this by incorporating program execution information. DATAVINCI outperforms 11 baselines across 4 benchmarks.

References

- [1] Hiba Abu Ahmad and Hongzhi Wang. 2020. Automatic weighted matching rectifying rule discovery for data repairing: Can we discover effective repairing rules automatically from dirty data? *The VLDB Journal* 29, 6 (2020), 1433–1447.
- [2] Djamel Belazzougui and Mathieu Raffinot. 2011. Approximate regular expression matching with multi-strings. In *International symposium on string processing and information retrieval*. Springer, 55–66.
- [3] Ranjita Bhagwan, Sonu Mehta, Arjun Radhakrishna, and Sahil Garg. 2021. Learning Patterns in Configuration. In *36th IEEE/ACM International Conference on Automated Software Engineering, ASE 2021, Melbourne, Australia, November 15-19, 2021*. IEEE, 817–828. <https://doi.org/10.1109/ASE51524.2021.9678525>
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. <https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf>
- [5] Jean-Marc Champarnaud, Hadrien Jeanne, and Ludovic Mignot. 2013. Derivatives of approximate regular expressions. *Discrete Mathematics & Theoretical Computer Science* 15, Automata, Logic and Semantics (2013).
- [6] Qiaochu Chen, Arko Banerjee, Çağatay Demiralp, Greg Durrett, and Işıl Dillig. 2023. Data extraction via semantic regular expression synthesis. *Proceedings of the ACM on Programming Languages* 7, OOPSLA2 (2023), 1848–1877.
- [7] Qiaochu Chen, Arko Banerjee, Çağatay Demiralp, Greg Durrett, and Işıl Dillig. 2023. Data Extraction via Semantic Regular Expression Synthesis. *Proceedings of the ACM on Programming Languages* 7, OOPSLA2 (2023), 1848–1877.
- [8] Xu Chu, Ihab F Ilyas, Sanjay Krishnan, and Jiannan Wang. 2016. Data cleaning: Overview and emerging challenges. In *Proceedings of the 2016 international conference on management of data*. 2201–2206.
- [9] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2009. *Introduction to Algorithms, Third Edition* (3rd ed.). The MIT Press.
- [10] Alireza Heidari, Joshua McGrath, Ihab F Ilyas, and Theodoros Rekatsinas. 2019. Holodetect: Few-shot learning for error detection. In *Proceedings of the 2019 International Conference on Management of Data*. 829–846.
- [11] Zhipeng Huang and Yeye He. 2018. Auto-detect: Data-driven error detection in tables. In *Proceedings of the 2018 International Conference on Management of Data*. 1377–1392.
- [12] Madelon Hulsebos, Kevin Hu, Michiel Bakker, Emanuel Zraggen, Arvind Satyanarayan, Tim Kraska, Çağatay Demiralp, and César Hidalgo. 2019. Sherlock: A Deep Learning Approach to Semantic Data Type Detection. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM.
- [13] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. *arXiv:2310.06825 [cs.CL]* <https://arxiv.org/abs/2310.06825>
- [14] Sanjay Krishnan, Jiannan Wang, Eugene Wu, Michael J Franklin, and Ken Goldberg. 2016. Activeclean: Interactive data cleaning for statistical modeling. *Proceedings of the VLDB Endowment* 9, 12 (2016), 948–959.

- [15] Yuanzhi Li, Sébastien Bubeck, Ronen Eldan, Allie Del Giorno, Suriya Gunasekar, and Yin Tat Lee. 2023. Textbooks Are All You Need II: **phi-1.5** technical report. *arXiv preprint arXiv:2309.05463* (2023).
- [16] Mohammad Mahdavi and Ziawasch Abedjan. 2020. Baran: Effective error correction via a unified context representation and transfer learning. *Proceedings of the VLDB Endowment* 13, 12 (2020), 1948–1961.
- [17] Mohammad Mahdavi and Ziawasch Abedjan. 2021. Semi-Supervised Data Cleaning with Raha and Baran.. In *CIDR*.
- [18] Mohammad Mahdavi, Ziawasch Abedjan, Raul Castro Fernandez, Samuel Madden, Mourad Ouzzani, Michael Stonebraker, and Nan Tang. 2019. Raha: A configuration-free error detection system. In *Proceedings of the 2019 International Conference on Management of Data*. 865–882.
- [19] Microsoft. 2015. Program synthesis from input-output examples (PROSE). <https://microsoft.github.io/prose>.
- [20] Eugene W Myers and Webb Miller. 1989. Approximate matching of regular expressions. *Bulletin of mathematical biology* 51, 1 (1989), 5–37.
- [21] Avanika Narayan, Ines Chami, Laurel Orr, Simran Arora, and Christopher Ré. 2022. Can Foundation Models Wrangle Your Data? *arXiv preprint arXiv:2205.09911* (2022).
- [22] Gonzalo Navarro. 2001. A guided tour to approximate string matching. *ACM computing surveys (CSUR)* 33, 1 (2001), 31–88.
- [23] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Lukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Lukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kopic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O’Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolai Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. 2024. GPT-4 Technical Report. *arXiv:2303.08774 [cs.CL]* <https://arxiv.org/abs/2303.08774>
- [24] Saswat Padhi, Prateek Jain, Daniel Perelman, Oleksandr Polozov, Sumit Gulwani, and Todd Millstein. 2018. FlashProfile: a framework for synthesizing data profiles. *Proceedings of the ACM on Programming Languages* 2, OOPSLA (2018), 1–28.

- [25] Jun-U Park, Sang-Ki Ko, Marco Cognetta, and Yo-Sub Han. 2019. SoftRegex: Generating Regex from Natural Language Descriptions using Softened Regex Equivalence. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (Eds.). Association for Computational Linguistics, Hong Kong, China, 6425–6431. <https://doi.org/10.18653/v1/D19-1677>
- [26] Abdulhakim Qahtan, Nan Tang, Mourad Ouzzani, Yang Cao, and Michael Stonebraker. 2020. Pattern functional dependencies for data cleaning. (2020).
- [27] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research* 21, 140 (2020), 1–67. <http://jmlr.org/papers/v21/20-074.html>
- [28] Kia Rahmani, Mohammad Raza, Sumit Gulwani, Vu Le, Daniel Morris, Arjun Radhakrishna, Gustavo Soares, and Ashish Tiwari. 2021. Multi-Modal Program Inference: A Marriage of Pre-Trained Language Models and Component-Based Synthesis. *Proc. ACM Program. Lang.* 5, OOPSLA, Article 158 (oct 2021), 29 pages. <https://doi.org/10.1145/3485535>
- [29] Kia Rahmani, Mohammad Raza, Sumit Gulwani, Vu Le, Daniel Morris, Arjun Radhakrishna, Gustavo Soares, and Ashish Tiwari. 2021. Multi-modal program inference: a marriage of pre-trained language models and component-based synthesis. *Proc. ACM Program. Lang.* 5, OOPSLA, Article 158 (Oct. 2021), 29 pages. <https://doi.org/10.1145/3485535>
- [30] Vijayshankar Raman and Joseph M Hellerstein. 2001. Potter’s wheel: An interactive data cleaning system. In *VLDB*, Vol. 1. 381–390.
- [31] Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, and Christopher Ré. 2017. HoloClean: Holistic Data Repairs with Probabilistic Inference. *Proc. VLDB Endow.* 10, 11 (aug 2017), 1190–1201. <https://doi.org/10.14778/3137628.3137631>
- [32] Mukul Singh, José Cambronero, Sumit Gulwani, Vu Le, Carina Negreanu, Mohammad Raza, and Gust Verbruggen. 2022. CORNET: A neurosymbolic approach to learning conditional table formatting rules by example. *arXiv preprint arXiv:2208.06032* (2022).
- [33] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv:2307.09288 [cs.CL]* <https://arxiv.org/abs/2307.09288>
- [34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. <https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>
- [35] Gust Verbruggen, Vu Le, and Sumit Gulwani. 2021. Semantic programming by example with pre-trained models. *Proceedings of the ACM on Programming Languages* 5, OOPSLA (2021), 1–25.
- [36] Pei Wang and Yeye He. 2019. Uni-detect: A unified approach to automated error detection in tables. In *Proceedings of the 2019 International Conference on Management of Data*. 811–828.
- [37] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 8696–8708. <https://doi.org/10.18653/v1/2021.emnlp-main.685>

Received July 2024; revised September 2024; accepted November 2024